

Roteiro do Vídeo – ProjetoEscolaEAD

Duração estimada: 7 a 8 minutos · O vídeo precisa ter narração · As falas são sugestões: diga com suas palavras.

Antes de gravar

- Feche e abra o projeto no BlueJ de novo e confira se as **6 classes** aparecem no diagrama. Se faltar alguma, use *Editar → Adicionar Classe a partir de Arquivo*.
- Aumente a fonte do BlueJ e do terminal para ficar legível no vídeo.
- Faça um ensaio rápido e confira se o microfone está gravando.
- Deixe anotados os dados de teste abaixo.

Dados de teste

Item	Dados
Aluno 1	código 1, Carlos, não bolsista, 3 parcelas de 500
Aluno 2	código 2, Ana, bolsista "Integral", 12 parcelas de 450,50
Cursos	Java Basico (40h) e Python (30h)

Roteiro

1 Abertura

~20s

TELA BlueJ com o diagrama de classes.

FALA

Este é o ProjetoEscolaEAD, um sistema de console para uma escola EAD feito em Java com orientação a objetos. Ele tem seis classes: Aluno, AlunoBolsista, Curso, ListaDeAlunos, Mensalidade e SistemaEscolaEAD, que tem o menu principal.

2 Encapsulamento

~30s

TELA Aluno.java: atributos (linhas 4–16) e um par get/set.

FALA

Todos os atributos são private, então só a própria classe mexe neles diretamente. Quem está de fora usa os getters e setters. Isso é o encapsulamento, e todas as classes do projeto seguem esse padrão.

3 Herança e sobrescrita

~40s

TELA `AlunoBolsista.java`.

FALA

O `AlunoBolsista` usa `extends Aluno`, ou seja, herda tudo do `Aluno` e acrescenta o atributo `tipoBolsa`. No construtor, o `super` chama o construtor do `Aluno` para preencher os dados comuns. O `exibeDados` foi sobrescrito: ele chama a versão do `Aluno` com `super.exibeDados()` e depois mostra o tipo de bolsa.

4 Menu: do-while e switch

~40s

TELA `SistemaEscolaEAD.java`, linhas 35–68.

FALA

O menu fica dentro de um `do-while`, porque ele precisa aparecer pelo menos uma vez e se repete até o usuário escolher 3, que é `Sair`. O `switch` decide o que fazer em cada opção.

AÇÃO Rodar o `main` e mostrar o menu.

5 Opção 2: adicionar alunos e polimorfismo

~1min

TELA `SistemaEscolaEAD.java`, linhas 100–108.

FALA

Aqui tem polimorfismo: a variável é do tipo `Aluno`, mas pode guardar um `AlunoBolsista`, porque um bolsista também é um aluno.

AÇÃO Cadastrar Carlos (não bolsista) e Ana (bolsista).

AÇÃO Tentar cadastrar de novo com o código 1.

FALA

Se eu tento usar um código que já existe, o sistema bloqueia.

6 Opção 1: lista de alunos e array unidimensional

~45s

TELA `ListaDeAlunos.java`: linha 4 e o laço do `exibirLista`.

FALA

Os alunos ficam num array unidimensional, o `Aluno[]`. O `adicionarAluno` usa um `for` para verificar se o código já existe. O `exibirLista` percorre o vetor chamando `exibeDados()`. Como a Ana é bolsista, o Java roda automaticamente a versão sobrescrita, e por isso aparece o tipo de bolsa só para ela. Isso também é polimorfismo.

AÇÃO Opção 1.

7 Opção 4: notas

~1min

TELA Aluno.java, linhas 11–12, e o calcularMedia (linha 96).

FALA

Cada aluno tem um vetor `double[3]` com as notas e um `boolean[3]` que marca quais já foram lançadas. O `calcularMedia` soma as três com um `for` e divide por 3.

AÇÃO Opção 4, código 1: lançar a nota 1 = 8,5 e a nota 3 = 7.

AÇÃO Opção 4 com o código 99.

FALA

Antes de mostrar as notas, o sistema confere se o aluno existe.

8 Opção 5: financeiro

~1min

TELA Mensalidade.java e o pagarMensalidade em Aluno.java (linha 148).

FALA

Cada mensalidade tem um valor e um status de pago. O aluno guarda suas mensalidades num array de `Mensalidade`, e o `darBaixa()` marca a parcela como paga.

AÇÃO Opção 5, código 2: pagar a parcela 3.

AÇÃO Tentar pagar a parcela 3 de novo para mostrar o aviso “já está paga”.

9 Opções 6, 7 e 8: cursos e array bidimensional

~1min30

TELA SistemaEscolaEAD.java, linha 22.

FALA

Este é o array bidimensional. Funciona como uma tabela: cada linha é um curso e cada coluna é uma vaga de aluno naquele curso.

AÇÃO Opção 6: cadastrar os 2 cursos.

AÇÃO Opção 7: matricular Carlos e Ana no curso 1. Na Ana, digitar primeiro uma opção errada.

FALA

Na escolha do curso usei um `do-while` que repete até a opção ser válida.

AÇÃO Opção 7 de novo, com o Carlos no mesmo curso, para mostrar o aviso “já está matriculado”.

AÇÃO Opção 8.

FALA

Um `for` percorre os cursos e, dentro dele, outro `for` percorre a linha da matriz para listar os alunos de cada curso.

10 Encerramento

~30s

AÇÃO Opção 3 (Sair).

FALA

Para resumir, o projeto usa encapsulamento com atributos `private` e `getters` e `setters`, herança e polimorfismo entre `Aluno` e `AlunoBolsista`, arrays unidimensionais em `alunos`, `notas` e `mensalidades`, um array bidimensional na matriz de matrículas, as repetições `for`, `while` e `do-while`, e as seleções `if` e `switch`.

Checklist: conceitos exigidos pelo PDF

✓	Conceito	Onde aparece	Cena
☐	Encapsulamento	<code>private</code> , <code>getters</code> e <code>setters</code> em todas as classes	2
☐	Herança e polimorfismo	<code>Aluno</code> → <code>AlunoBolsista</code> (<code>super</code> , <code>@Override</code>)	3, 5, 6
☐	Array unidimensional	<code>alunos[]</code> , <code>notas[]</code> , <code>mensalidades[]</code>	6, 7, 8
☐	Array bidimensional	<code>Aluno[][]</code> <code>matriculas</code>	9
☐	Estruturas de repetição	<code>for</code> , <code>while</code> , <code>do-while</code>	4, 6, 7, 9
☐	Estruturas de seleção	<code>if</code> , <code>switch</code>	4, 7, 8, 9

Publicação

- **YouTube:** publique como "Não listado", para que qualquer pessoa com o link consiga ver.
- **Google Drive:** compartilhe com "Qualquer pessoa com o link".
- Antes de entregar, abra o link numa janela anônima para confirmar que o vídeo abre.